AIRLINE BOOKING SYSTEM

Team Members

Snikitha	2510030305
Nandini	2510030283
Varshitha	2510030400
SriHarini	2510030374
Deekshitha	2510080059
Anushka	2510030035
Naga Sreeja	2510030173
Varna	2510080025

Project Abstract

Defining Goals for the Booking System

The main objectives include: enhancing user experience through intuitive design, providing efficient flight management, and implementing effective algorithms for searching and sorting flight data seamlessly.

Modules

Flight Management Module

Stores flight details (Flight No, From, To, Time, Airline, Date, Cost, Seats)
Uses Singly Linked List
Functions:
Add Flight
Display Flights
Convert list to array (for sorting)

Booking Management Module

Stores passenger booking details
Uses Doubly Linked List
Functions:
Add Booking
Display Bookings

Check-in Queue Module

Manages passenger check-in process
Uses Queue (FIFO)
Functions:
Join Queue
Process Check-in
Display Queue

Searching Module

Uses Generic Binary Search
Searches flights efficiently
Based on Comparable interface

Sorting Module

Quick Sort
Sorts flights by flight number
Merge Sort
Sorts flights by departure time

File Handling Module

Reads & writes data to files:
flights.txt
bookings.txt
queue.txt
Ensures data persistence

Advanced Search Module

Search flights by:
Source & Destination
Date Range (± days)
Uses Date & Time calculations

Description of modules

Flight Management Module

This module handles all flight-related operations in the system. It stores details like flight number, source, destination, departure time, airline, date, cost, and available seats. A singly linked list is used to manage flight records dynamically.
Users can add new flights, view all flights in a structured table format, and convert the list into an array for sorting purposes. This module forms the core data storage of the system.

Booking Management Module

This module manages passenger bookings. It stores the passenger name and corresponding flight number using a doubly linked list, allowing easy traversal in both directions.
Users can add new bookings and view all existing bookings. This structure helps efficiently maintain and organize booking records.

Check-in Queue Module

This module simulates the airport check-in process using a queue (FIFO principle). Passengers are added to the queue when they arrive for check-in and are processed in order.
It includes operations like joining the queue, processing check-in, and displaying the current queue.
This ensures fair and real-time handling of passengers.

Searching Module

This module uses a generic binary search algorithm to efficiently search flight records. It works on sorted data and reduces search time compared to linear search.
It is implemented using Java generics, making it reusable for different data types.

Sorting Module

Sorting is implemented using two algorithms:

Quick Sort: Sorts flights based on flight number

Merge Sort: Sorts flights based on departure time
These algorithms improve data organization and help users view flights in a structured manner.

Data structures used

Data Structure	Where Used	Purpose / Description
Singly Linked List	Flight Management	Stores and manages flight records dynamically; allows easy insertion and traversal.
Doubly Linked List	Booking Management	Stores booking details with forward and backward traversal for efficient data handling.
Queue (FIFO)	Check-in Module	Manages passenger check-in process in order of arrival (First In First Out).
Array	Sorting & Searching	Used to convert linked list data for applying sorting and binary search algorithms.
Generic Class	Binary Search	Enables reusable searching logic for different data types using Comparable interface.

```java
    case 1:
        System.out.print("Flight No: ");
        String no = sc.nextLine();
        System.out.print("From: ");
        String from = sc.nextLine();
        System.out.print("To: ");
        String to = sc.nextLine();
        System.out.print("Departure Time: ");
        String time = sc.nextLine();
        System.out.print("Airline: ");
        String airline = sc.nextLine();
        System.out.print("Date (DD-MM-YYYY): ");
        String date = sc.nextLine();
        System.out.print("Cost: ");
        double cost = sc.nextDouble();
        System.out.print("Seats: ");
        int seats = sc.nextInt();
        sc.nextLine();

        flightList.addFlight(new Flight(no, from, to, time, airline, date, cost, seats));
        saveFlightsToFile();
        System.out.println("Flight Added!");
        break;

    case 2:
        flightList.displayTable();
        break;

    case 3:
        Flight[] qarr = flightList.toArray();
        new QuickSort<Flight>().quickSort(qarr,0,qarr.length-1);
        System.out.println("Flights Sorted By Flight Number:");
        System.out.println("-----------------------------------------------------------------------------------------");
        System.out.printf("%-8s %-10s %-10s %-10s %-12s %-12s %-8s %-6s\n",
                "Flight","From","To","Time","Airline","Date","Cost","Seats");
        System.out.println("-----------------------------------------------------------------------------------------");
        for(Flight f : qarr){
            System.out.printf("%-8s %-10s %-10s %-10s %-12s %-12s %-8.0f %-6d\n",
                    f.number,f.from,f.to,f.depTime,f.airline,f.date,f.cost,f.seats);
        }
        System.out.println("-----------------------------------------------------------------------------------------");
        break:
```

```java
            }
            System.out.println("-------------------------------------------------------------------------------------");
            break;

        case 4:
            Flight[] marr = flightList.toArray();
            new MergeSort().mergeSort(marr,0,marr.length-1);
            System.out.println("Flights Sorted By Time:");
            System.out.println("-------------------------------------------------------------------------------------");
            System.out.printf("%-8s %-10s %-10s %-10s %-12s %-12s %-8s %-6s\n",
                    "Flight","From","To","Time","Airline","Date","Cost","Seats");
            System.out.println("-------------------------------------------------------------------------------------");
            for(Flight f : marr){
                System.out.printf("%-8s %-10s %-10s %-10s %-12s %-12s %-8.0f %-6d\n",
                        f.number,f.from,f.to,f.depTime,f.airline,f.date,f.cost,f.seats);
            }
            System.out.println("-------------------------------------------------------------------------------------");
            break;

        case 5:
            System.out.print("Passenger Name: ");
            String name=sc.nextLine();
            System.out.print("Flight No: ");
            String fno=sc.nextLine();
            bookingList.addBooking(new Booking(name,fno));
            saveBookingsToFile();
            System.out.println("Booking Added!");
            break;

        case 6:
            bookingList.displayTable();
            break;

        case 7:
            System.out.print("Passenger Name: ");
            String pname=sc.nextLine();
            checkQueue.joinQueue(pname);
            saveQueueToFile();
            break;
```

```java
case 10:

    // --- Search Flights by Route & Date Range ---
    System.out.print("From: ");
    String fromPlace = sc.nextLine();
    System.out.print("To: ");
    String toPlace = sc.nextLine();
    System.out.print("Enter Date (DD-MM-YYYY): ");
    String startDateStr = sc.nextLine();
    System.out.print("Enter number of days range (+/-): ");
    int dayRange = sc.nextInt();
    sc.nextLine();

    SimpleDateFormat sdf2 = new SimpleDateFormat("dd-MM-yyyy");
    Date startDate = sdf2.parse(startDateStr);

    FlightNode tempNode = flightList.head;

    System.out.println("\nFlights available from " + fromPlace + " to " + toPlace + " within " + dayRange + " days:\n");
    System.out.println("-------------------------------------------------------------------------------------");
    System.out.printf("%-8s %-10s %-10s %-10s %-12s %-12s %-8s %-6s\n",
            "Flight","From","To","Time","Airline","Date","Cost","Seats");
    System.out.println("-------------------------------------------------------------------------------------");

    while(tempNode != null){
        Flight f = tempNode.data;
        Date flightDate = sdf2.parse(f.date);
        long diff = Math.abs(flightDate.getTime() - startDate.getTime());
        long diffDays = TimeUnit.DAYS.convert(diff, TimeUnit.MILLISECONDS);

        if(f.from.equalsIgnoreCase(fromPlace) && f.to.equalsIgnoreCase(toPlace) && diffDays <= dayRange){
            System.out.printf("%-8s %-10s %-10s %-10s %-12s %-12s %-8.0f %-6d\n",
                    f.number,f.from,f.to,f.depTime,f.airline,f.date,f.cost,f.seats);
        }

        tempNode = tempNode.next;
    }
    System.out.println("-------------------------------------------------------------------------------------");
    break;
```

```java
// ---------------- CHECK-IN QUEUE ----------------
class CheckInQueue{
    Queue<String> queue=new LinkedList<>();

    void joinQueue(String name){
        queue.add(name);
        System.out.println(name+" joined check-in queue.");
    }

    void processCheckIn(){
        if(queue.isEmpty()){
            System.out.println("No passengers in queue.");
            return;
        }
        String p=queue.remove();
        System.out.println(p+" check-in completed.");
    }

    void displayTable(){
        if(queue.isEmpty()){
            System.out.println("Queue is empty.");
            return;
        }

        System.out.println("----------------------------");
        System.out.printf("%-20s\n","Passenger Name");
        System.out.println("----------------------------");
        for(String p:queue)
            System.out.printf("%-20s\n",p);
        System.out.println("----------------------------");
    }
}
```

sample output

```
=================== AIRLINE BOOKING SYSTEM ===================
S.No   Description
---------------------------------------
1      Add Flight
2      Display Flights
3      Quick Sort Flights (By Flight Number)
4      Merge Sort Flights (By Time)
5      Add Booking
6      Display Bookings
7      Join Check-in Queue
8      Process Check-in
9      Display Check-in Queue
10     Search Flights by Route & Date Range
11     Exit
===============================================================
Enter choice:
```

```
-----------------------------------------------------------------
Enter choice: 10
From: DEL
To: USA
Enter Date (DD-MM-YYYY): 21-04-2026
Enter number of days range (+/-): 3

Flights available from DEL to USA within 3 days:

-----------------------------------------------------------------------------------------
Flight   From       To         Time       Airline      Date         Cost     Seats
-----------------------------------------------------------------------------------------
AI108    DEL        USA        09:50AM    AirIndia     20-04-2026   25000    154
1221     DEL        USA        3.00AM     Indigo       22-04-2026   30000    300
```

Display

```
==============================================================
Enter choice: 2
-------------------------------------------------------------------------------------
Flight    From       To         Time       Airline      Date         Cost     Seats
-------------------------------------------------------------------------------------
AI104     HYD        DEL        08:30AM    AirIndia     12-04-2026   5200     100
6E201     HYD        DEL        10:00AM    Indigo       12-04-2026   5000     120
AI105     DEL        BOM        10:15AM    AirIndia     14-04-2026   4600     250
6E305     DEL        BOM        01:00PM    Indigo       15-04-2026   4400     200
AI106     BOM        BLR        01:45PM    AirIndia     16-04-2026   4950     234
6E410     BOM        BLR        03:30PM    Indigo       17-04-2026   4700     180
AI107     BLR        MAA        06:20PM    AirIndia     18-04-2026   4100     435
6E520     BLR        MAA        07:10PM    Indigo       19-04-2026   3900     300
AI108     MAA        HYD        09:50AM    AirIndia     20-04-2026   4300     154
6E610     MAA        HYD        11:00AM    Indigo       21-04-2026   4100     220
AI107     HYD        LHR        06:20PM    AirIndia     18-04-2026   48000    435
6E520     BOM        USA        07:10PM    Indigo       19-04-2026   20000    300
AI108     DEL        USA        09:50AM    AirIndia     20-04-2026   25000    154
6E610     BOM        LHR        11:00AM    Indigo       21-04-2026   41000    220
AI108     DEL        CND        09:50AM    AirIndia     20-04-2026   42300    154
6E610     DEL        DUB        11:00AM    Indigo       21-04-2026   40100    220
1221      MUM        USA        3.00AM     Indigo       12-04-2026   30000    300
-------------------------------------------------------------------------------------
```

What we learned?

Through this project, we gained practical knowledge of applying data structures and algorithms in real-world applications.
We learned how to use different data structures like linked lists, queues, and arrays to efficiently manage and organize data.
We understood the implementation of sorting algorithms (Quick Sort, Merge Sort) and searching techniques (Binary Search) to improve performance.
We also gained experience in file handling, which helped us store and retrieve data permanently.
Additionally, we improved our skills in problem-solving, logical thinking, and code organization.
Working as a team helped us develop communication, collaboration, and project management skills.

Future Analysis

This project can be further improved by adding more advanced features and technologies.
Graphical User Interface (GUI): Replace console-based system with a user-friendly interface using JavaFX or web technologies.
Database Integration: Use databases like MySQL instead of text files for better data security and scalability.
User Authentication: Add login/signup system for passengers and admin.
Online Booking System: Enable real-time booking through a web or mobile application.
Seat Selection Feature: Allow users to choose seats visually.
Payment Integration: Add online payment options for booking confirmation.
Advanced Search & Filters: Include filters like price range, airline, and time.
Notifications: Send booking confirmations via email or SMS.

Thank you.

We also extend our heartfelt thanks to our faculty members for their valuable suggestions and constant motivation, which helped us improve and successfully complete this project.